

Python Code

```
1  """
2  Map Search
3  """
4
5  import comp140_module7 as maps
6
7  class Queue:
8      """
9      A simple implementation of a FIFO queue.
10     """
11
12     def __init__(self):
13         self._queue = []
14
15     def __len__(self):
16         return len(self._queue)
17
18     def __str__(self):
19         return str(self._queue)
20
21     def push(self, item):
22         """
23         Adds an item to the back of the queue.
24         inputs:
25             - item: the item to be added to the queue
26         Returns: nothing
27         """
28         self._queue.append(item)
29
30     def pop(self):
31         """
32         Removes and returns the item at the front of the queue.
33         inputs: nothing
34         Returns: the item at the front of the queue
35         """
36         if len(self._queue) == 0:
37             raise IndexError("empty queue")
38         return self._queue.pop(0)
39
40     def clear(self):
41         """
42         Removes all items from the queue.
43         inputs: nothing
44         Returns: nothing
45         """
46         self._queue = []
47
48     class Stack:
49         """
50         A simple implementation of a LIFO stack.
```

```
51     """
52     def __init__(self):
53         self._stack = []
54
55     def __len__(self):
56         return len(self._stack)
57
58     def __str__(self):
59         return str(self._stack)
60
61     def push(self, item):
62         """
63         Adds an item to the top of the stack.
64         inputs:
65             - item: the item to be added to the stack
66         Returns: nothing
67         """
68         self._stack.append(item)
69
70     def pop(self):
71         """
72         Removes and returns the item at the top of the stack.
73         inputs: nothing
74         Returns: the item at the top of the stack
75         """
76         if len(self._stack) == 0:
77             raise IndexError("empty stack")
78         return self._stack.pop()
79
80     def clear(self):
81         """
82         Removes all items from the stack.
83         inputs: nothing
84         Returns: nothing
85         """
86         self._stack = []
87
88
89     def bfs_dfs(graph, rac_class, start_node, end_node):
90         """
91         Performs a breadth-first search or a depth-first search on graph
92         starting at the start_node. The rac_class should either be a
93         Queue class or a Stack class to select BFS or DFS.
94
95         Completes when end_node is found or entire graph has been
96         searched.
97
98         inputs:
99             - graph: a directed Graph object representing a street map
100             - rac_class: a restricted access container (Queue or Stack) class to
101               use for the search
102             - start_node: a node in graph representing the start
103             - end_node: a node in graph representing the end
104
```

```
105 Returns: a dictionary associating each visited node with its parent
106 node.
107 """
108 # initialize
109 dist = {}
110 parent = {}
111
112 for node in graph.nodes():
113     dist[node] = float('inf')
114     parent[node] = None
115
116 # start
117 dist[start_node] = 0
118
119 # RAC:queue or stack
120 rac = rac_class()
121 rac.push(start_node)
122
123 # iterate over graph
124 while len(rac) > 0:
125     node = rac.pop()
126
127     # stop early if end is found
128     if node == end_node:
129         return parent
130
131     # iterate over neighbors
132     for nbr in graph.get_neighbors(node):
133         if dist[nbr] == float('inf'):
134             dist[nbr] = dist[node] + 1
135             parent[nbr] = node
136             rac.push(nbr)
137
138 return parent
139
140 def dfs(graph, start_node, end_node, parent):
141     """
142     Performs a recursive depth-first search on graph starting at the
143     start_node.
144
145     Completes when end_node is found or entire graph has been
146     searched.
147
148     inputs:
149         - graph: a directed Graph object representing a street map
150         - start_node: a node in graph representing the start
151         - end_node: a node in graph representing the end
152         - parent: a dictionary that initially has one entry associating
153             the original start_node with None
154
155     Modifies the input parent dictionary to associate each visited node
156     with its parent node
157     """
158     if start_node == end_node:
```

```

159         return
160
161     # iterate over neighbors
162     for nbr in graph.get_neighbors(start_node):
163         #unvisited
164         if nbr not in parent:
165             # update parent
166             parent[nbr] = start_node
167             # recursion
168             dfs(graph, nbr, end_node, parent)
169
170         # end node found early
171         if end_node in parent:
172             return
173
174     return
175
176 def astar(graph, start_node, end_node,
177          edge_distance, straight_line_distance):
178     """
179     Performs an A* search on graph starting at start_node.
180
181     Completes when end_node is found or entire graph has been
182     searched.
183
184     inputs:
185     - graph: a directed Graph object representing a street map
186     - start_node: a node in graph representing the start
187     - end_node: a node in graph representing the end
188     - edge_distance: a function which takes two nodes and a graph
189                     and returns the actual distance between two
190                     neighboring nodes
191     - straight_line_distance: a function which takes two nodes and
192                             a graph and returns the straight line distance
193                             between two nodes
194
195     Returns: a dictionary associating each visited node with its parent
196     node.
197     """
198     #initialize
199     open_set = set([start_node])
200     parent = {start_node: None}
201
202     # distance from start_node to each node
203     g_value = {start_node: 0}
204
205     # heuristic + distance
206     f_value = {
207         start_node: straight_line_distance(start_node, end_node, graph)
208     }
209
210     while len(open_set) > 0:
211         # find minimm f val
212         current = min(open_set, key=lambda node: f_value[node])

```

```
213
214     if current == end_node:
215         return parent
216
217     open_set.remove(current)
218
219     # iterate
220     for nbr in graph.get_neighbors(current):
221         tent_g = g_value[current] + edge_distance(current, nbr, graph)
222
223         # update g and f values
224         if nbr not in g_value or tent_g < g_value[nbr]:
225             g_value[nbr] = tent_g
226             f_value[nbr] = tent_g + straight_line_distance(nbr, end_node, graph)
227             parent[nbr] = current
228             open_set.add(nbr)
229
230     return parent
231
232 # You can replace functions/classes you have not yet implemented with
233 # None in the call to "maps.start" below and the other elements will
234 # work.
235
236 maps.start(bfs_dfs, Queue, Stack, dfs, astar)
237
```